



COMSATS University
Islamabad, Sahiwal Campus

Keep & Lock
(Web-based Application)

A Project Represented to
COMSATS University Islamabad, Sahiwal Campus

In partial fulfillment
the requirement of the degree of

Bachelor of Science in Software Engineering (2020-2024)

By

Raeesa Mukhtar	CIIT/ SP20-BSE-019 /SWL
Sana Siddique	CIIT/ SP20-BSE-023 /SWL
Zunaira Rasheed	CIIT/ SP20-BSE-030 /SWL



COMSATS University
Islamabad, Sahiwal Campus

Keep & Lock
(Web-based Application)

By

Raeesa Mukhtar	CIIT/ SP20-BSE-019 /SWL
Sana Siddique	CIIT/ SP20-BSE-023 /SWL
Zunaira Rasheed	CIIT/ SP20-BSE-030 /SWL

Supervisor

Mr. Ahmad Shaf

Bachelor of Science in Software Engineering (2020-2024)

The candidates confirm that the work submitted on their own and appropriate credit has been given where reference has been made to the work of oth.

DECLARATION

We hereby declare that this website, neither whole nor as a part has been copied out from any source. It is further declared that we have developed this website and accompanied report entirely based on our personal efforts. If any part of this project is proved to be copied out from any source or found to be a reproduction of some other. We will stand by the consequences. No portion of the work presented has been submitted because of any application for any other degree or qualification of this or any other university or institute of learning.

Raeesa Mukhtar

Sana Siddique

Zunaira Rasheed

CERTIFICATE OF APPROVAL

It is to certify that the final year project of BS(SE) “**Keep & Lock**” was developed by **Raeesa Mukhtar (CIIT/SP20-BSE-019/SWL)**, **Sana Siddique (CIIT/SP20-BSE-023/SWL)** and **Zunaira Rasheed (CIIT/SP20-BSE-030/SWL)** under the supervision of “**Mr. Ahmad Shaf**” and that in his opinion; it is fully adequate, in scope and quality for the degree of bachelors of science in Software Engineering.

Supervisor

Mr. Ahmad Shaf

Lecturer

Department of Computer Science

COMSATS University Islamabad, Sahiwal Campus

External Examiner

Dr. Muhammad Munawar Iqbal

Assistant Professor

Department of Computer Science

University of Engineering and Technology, Taxila

Head of Department

Dr. Tariq Ali

Department of Computer Science

COMSATS University Islamabad, Sahiwal Campus

Executive Summary

In Pakistan's thriving agricultural landscape, the effective storage of product, particularly potatoes, plays a pivotal role in preserving freshness and quality. However, the existing system lacks a holistic solution that seamlessly integrates the reservation of storage space and the purchase of goods. Our innovative project aims to create an online platform that revolutionizes the way customers interact with cold storage facilities, offering a comprehensive solution for storing and purchasing potatoes. This dynamic platform not only empowers customers to reserve storage space and place orders but also provides them with the option to buy, all while ensuring seamless administrative management.

Acknowledgment

All praise is to almighty Allah who bestowed upon us a minute portion of his boundless knowledge by which we were able to accomplish this challenging task.

We are greatly indebted to our project supervisor “**Mr. Ahmad Shaf**”. Without their supervision, advice, and valuable guidance, the completion of this project would have been doubtful. We are deeply indebted to them for their encouragement and continual help during this work.

And we are also thankful to our parents and family who have been a constant source of encouragement for us and brought us the values of honesty & hard work.

Raeesa Mukhtar

Sana Siddique

Zunaira Rasheed

Abbreviations

SQL	Structured Query Language
API	Application Programming Interface
UI	User Interface
DFD	Data Flow Diagram
UC	Use Case
IT	Information Technology
PDL	Procedural Description Language
FYP	Final Year Project
DB	Database

Table of Contents

1	Introduction	1
1.1	Project Brief	1
1.2	Relevance to the course module	2
1.2.1	Database System	2
1.2.2	Web Technologies	2
1.2.3	Software Requirement Engineering	2
1.2.4	Software Project Management	2
1.2.5	Human Computer Interaction	2
1.3	Background	2
1.4	Literature Review	3
1.4.1	Analysis Review from Literature Review	3
1.5	Methodology and Software Life Cycle	4
1.5.1	Design Methodology	4
1.5.2	The Rationale behind the Selected Methodology	5
2	Problem Definition	6
2.1	Problem Statement	6
2.2	Deliverables and Development Requirements	6
2.2.1	Deliverables	6
2.2.2	Development Requirements	7
3	Requirement analysis	8
3.1	Use Case Diagram	8
3.1.1	Use Case of User	8
3.1.2	Use Case of Admin	12
3.2	Functional requirements	14
3.3	Non-functional requirements	16
3.3.1	Usability	16
3.3.2	Performance	16
3.3.3	Reliability	16
3.3.4	Availability	16
3.4	System Requirements	16
3.4.1	Hardware Requirements	16
3.4.2	Software Requirement	16
4	Design and Architecture	17
4.1	System Architecture	17
4.2	Data Representation	17
4.2.1	Sequence Diagram	17

4.2.2	Class Diagram of Keep & Lock	22
4.2.3	Data Flow Diagram	23
4.2.4	Process Flow Representation.....	25
4.2.5	Entity Relationship Diagram	27
4.3	Design Models.....	28
4.3.1	Rapid Application Development	28
5	Implementation.....	30
5.1	Modules of the Keep & Lock.....	30
5.1.1	Sign-up Module	30
5.1.2	Login Module	31
5.1.3	Reserve Module	31
5.1.4	Ordering Module	31
5.2	External API's.....	31
5.3	User Interface	32
5.4	Screen Objects and Actions.....	36
6	Testing and Evaluation	37
6.1	Manual Testing.....	37
6.1.1	System Testing	38
6.1.2	Unit Testing	38
6.2	Integration Testing	40
6.3	Functional Testing.....	41
7	Conclusion and Future Work.....	43
7.1	Conclusion.....	43
7.2	Future Work	43
	References.....	44

List of Figures

Figure 1. 1 Rapid Application Development.....	5
Figure 3. 1 Use case of Reservation.....	9
Figure 3. 2 Use case of Admin.....	12
Figure 4. 1 Structure of MVT.....	17
Figure 4. 2 Sequence diagram of Order Product.....	19
Figure 4. 3 Sequence of Reservation.....	20
Figure 4. 4 Sequence of Admin manages Reservation.....	21
Figure 4. 5 Class diagram of Keep & Lock.....	23
Figure 4. 6 DFD level 0 for Keep & Lock.....	24
Figure 4. 7 DFD level 1 of Keep & Lock.....	24
Figure 4. 8 Process flow diagram of Admin console.....	25
Figure 4. 9 Process flow diagram of Customer console.....	26
Figure 4. 10 ERD diagram of Order Product.....	27
Figure 4. 11 ERD diagram of Reservation.....	28
Figure 4.12 Rapid Application Model	29
Figure 5.1 Interface Contact Us.....	33
Figure 5.2 Interface of Sign up.....	33
Figure 5.3 Selection of reserving & order product.....	34
Figure 5.4 Buy now page.....	34
Figure 5.5 Checkout.....	35
Figure 5.6 Interface of Reservation.....	35

List of Tables

Table 1.1 Literature Review.....	3
Table 3.1 Table of Use case Reservation.....	9
Table 3.2 Table of Use case Order Product.....	10
Table 3.3 Use case description of Product detail.....	13
Table 5.1 External APIs.....	32
Table 6.1 Manual Testing.....	34
Table 6.2 System testing.....	37
Table 6.3 Unit testing of sign up.....	38
Table 6.4 Unit testing of Login.....	39
Table 6.5 Unit testing of Reserving Space.....	39
Table 6.6 Integration Testing	40
Table 6.7 Functional Testing	41

Chapter 1

Introduction

1 Introduction

A cold storage warehouse is essential for maintaining the optimal temperature to safeguard temperature-sensitive products, thereby extending their shelf life. These refrigerated facilities, commonly used for storing perishables like fruits and vegetables, such as tomatoes and potatoes, serve a vital role in preserving goods that spoil quickly. Unfortunately, many cold storage facilities still rely on manual data entry, leading to challenges in data management and modification. This manual process is not only time-consuming but also prone to data inaccuracies and uncertainties.

To address these issues, there is a need for a web application accessible across all devices, which can efficiently manage warehouse operations. Such an application should cater to both customers and administrators, offering a range of features. Customers should be able to register, request space reservations, cancel bookings, place product orders, select products, specify quantities, cancel orders, and make online payments. Administrators, on the other hand, need comprehensive control over cold storage management, product inventory, employee management, and customer relations. The system should also facilitate employee management and provide insights into online payment records.

The application offers the convenience of generating advance bills and allowing customers to purchase products. This approach minimizes errors, saves time, enhances accuracy, and automatically updates data [1].

1.1 Project Brief

Keep & Lock, the ultimate web-based solution for all your storage and logistics needs. Our platform is designed to revolutionize the way you manage your products and their storage. With Keep & Lock, you can easily order products and reserve secure storage space, all from the convenience of your computer or mobile device.

It simplifies the process of ordering products. Whether you require fresh produce or specialized goods, our user-friendly interface allows you to browse, select, and order with just a few clicks. Say goodbye to the hassle of traditional purchasing methods; we provide a seamless online experience that ensures your products are just a click away.

Our platform offers a hassle-free solution for reserving cold storage space. If you need a safe and reliable location to store your temperature-sensitive items, Keep & Lock has you covered. You can effortlessly reserve storage space tailored to your needs, ensuring the preservation of your products under ideal conditions.

The system meticulously maintains records related to products, their types, subtypes, and product handling, including loading and unloading information. The system also provides insights into employee data and online payment records.

1.2 Relevance to the course module

The primary objective of this project is to enhance convenience and save time for individuals through the utilization of a web-based application. Consequently, it is intricately linked to the domain of software engineering. According to our project the following are the course modules relevance to my project:

1.2.1 Database System

As we have studied the course database, we have applied all the concepts of the database in this project. We have used many queries i.e., retrieve, insert, update, and delete records in the database. The whole project database is maintained using MySQL.

1.2.2 Web Technologies

Our project is a website which uses all the concepts we studied in web technologies. We've used Html and CSS to design our website.

1.2.3 Software Requirement Engineering

We've gathered data using techniques studied in this subject.

1.2.4 Software Project Management

As we have studied the course software project management, we have applied tactics of software project management in this project. The work breaks down the structure and the Gantt chart for this project has been developed using software engineering principles.

1.2.5 Human Computer Interaction

This course's concepts are used to make our project simple, attractive, and user-friendly.

1.3 Background

Cold storage presents significant advantages for storing items and prolonging the shelf life of perishable goods. In Pakistan, an agrarian nation, there exists a compelling necessity to preserve various commodities at precise temperature levels to preserve their freshness and longevity. Neglecting this critical aspect can lead to spoilage or an escalating risk of contamination within these commodities. While numerous cold storage facilities exist, unfortunately, there is currently no reliable web application that offers the convenience of online product reservations and order processing. Often, manual data storage methods prove

time-consuming when searching for records within a single entity. Furthermore, making alterations to data can be a cumbersome process within a manual system. Hence, it is imperative to develop a Cold Storage Web Application that can proficiently manage stock records, streamlining operations [2].

1.4 Literature Review

We have studied systems that are precisely not like these, but some of the modules are implemented in these app:

- Student Project. live

1.4.1 Analysis Review from Literature Review

System analysis is a critical approach for acquiring a profound comprehension of a system. To collect comprehensive information about the system, we can explore various resources like books, research articles, and theories. Furthermore, visiting pertinent sites can yield valuable insights. The following table outlines the weaknesses of the existing system and offers proposed solutions:

Table 1.1 Literature Review

Application Name	Weakness	Proposed Project Solution
• Student Project. live	• The application lacks robust security measures, making it vulnerable to potential data breaches. • The UI design needs improvement, as it is not user-friendly and can be confusing for users.	• This application provides security • It will have a better UI.
• Cold chain IQ	• This application has limited features. • It offers the temperature monitoring and report functionalities but not provide the facilities order and reservation.	• This application provides the facility of order product. • This application also provides the facility of online reservation.

• Fresh temp	• This software is low quality processes and high cost. • It is used for monitoring the temperature of stored potatoes but not the facility of order product.	• It provides the security and accuracy of data. • The ui of this application is easy to understand
--	--	--

1.5 Methodology and Software Life Cycle

The keep & lock project will follow a rapid application development methodology. This methodology can be effectively used in the development of web applications to expedite the development process and enhance.

1.5.1 Design Methodology

Software process models play a pivotal role in enhancing the design, project management, and product management aspects of software development. While there exist several methodologies for software development, the Rapid Application Development (RAD) model emerges as an especially accurate and suitable choice for specific applications, such as the one in question.

The RAD model is characterized by its iterative and prototyping-oriented approach. It promotes accelerated development using rapid prototyping and close collaboration between developers and stakeholders. This iterative nature allows for frequent feedback and adjustments, ensuring that the final product closely aligns with user requirements [3].

In the context of the given application, which presumably involves aspects like design, project management, and product management, the RAD model offers several advantages. First and foremost, it encourages rapid prototyping and user involvement, facilitating the fine-tuning of design elements to meet specific needs effectively. Furthermore, its iterative nature enables swift project management by breaking the development process into manageable chunks, thus improving project timelines and resource allocation.

The RAD model promotes a responsive and flexible approach. Changes in requirements or priorities can be accommodated relatively easily. This adaptability ensures that the final product is not only accurate but also highly suitable for the intended application.

In conclusion, the RAD model stands out as an ideal choice for applications that demand precision, swift project management, and product adaptability. Its iterative and prototyping-

focused approach aligns closely with the needs of projects involving design, project management, and product management, ultimately leading to enhanced outcomes and customer satisfaction.

In this figure, we depict the iterative and collaborative nature of the RAD model, highlighting its phases, including Requirements Planning, User Design, Construction, and Cutover.

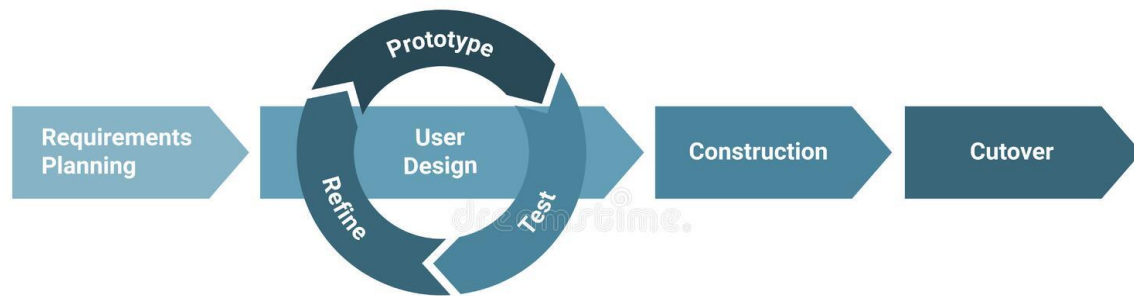


Figure 1. 1 Rapid Application Development

1.5.2 The Rationale behind the Selected Methodology

The Rapid Application Development model is an iterative approach to software development that shares similarities with the incremental methodology. Like incremental development, RAD also involves breaking down a substantial project into smaller, more manageable components or increments. Each increment is then developed and tested independently, fostering a collaborative environment that encourages feedback and continuous improvement throughout the development process.

For the Keep & Lock project, the RAD model was selected for several reasons. The iterative nature of RAD means that users can quickly test and provide feedback on the registration and login processes, ensuring a user-friendly experience. Password resets and account creation can be efficiently incorporated and improved upon with each iteration.

RAD facilitates efficient product registration and updates, ensuring a current product list. Customers can easily browse and select products, while administrators can promptly manage product details. RAD enables adjustments and timely notifications to users upon booking confirmation, providing a responsive booking experience for administrators.

In conclusion, the RAD model's adaptability, rapid prototyping, and iterative development approach align perfectly with the functional requirements of this system. It ensures that the application remains flexible, user-friendly, and responsive to evolving needs, ultimately leading to a robust and customer-centric solution.

Chapter 2

Problem Definition

2 Problem Definition

2.1 Problem Statement

In the current operational landscape, manual data storage and retrieval within a single entity have proven to be time-consuming and error-prone endeavors. The limitations of manual systems become evident, particularly when considering data modifications, where the complexities of updating records pose significant challenges. Regrettably, a comprehensive software solution that caters to the unique needs of both Cold Storage Owners and customers remains conspicuously absent from the market.

Our innovative web application has been meticulously designed with the express purpose of addressing these pressing issues. It empowers users with the ability to effortlessly input and manage various entities within the cold storage ecosystem. By transitioning to a digital framework, our aim is to streamline data management processes, enhance data accuracy. Additionally, our system extends its utility to administrative functions by efficiently storing crucial information about the workforce. Administrators gain the capability to access comprehensive insights into the workforce's performance and details.

2.2 Deliverables and Development Requirements

2.2.1 Deliverables

Deliverables are the concrete results or items that a project must accomplish and provide to the stakeholders.

2.2.1.1 Admin Module

Administrators can update product records. The admin module provides comprehensive control over customer information, employee management, product administration and space reservation [4].

2.2.1.2 Reservation Module

The reservation module in the web application enables users to request a reservation for a space in the cold store. Users initiate the process by sending a reservation request to the administrator. Upon receiving the request, the administrator reviews and evaluates it. If the request is accepted, the user is granted permission to proceed with the reservation, securing their designated space in the cold store. This streamlined process ensures efficient communication between users and administrators, facilitating a seamless experience for users to reserve storage space based on their needs.

2.2.1.3 Order Product

There is a user-friendly order module that empowers users to seamlessly place orders for their desired products. Users have the convenience of selecting products, adding them to their cart, and proceeding to a hassle-free checkout process. The system facilitates online payments, allowing users to securely and efficiently complete transactions. Additionally, users can review and confirm their orders before finalizing the purchase. The integration of a cohesive order and payment system enhances the overall user experience, providing a straightforward and efficient way for users to acquire products on the platform.

2.2.1.4 Feedback Module

Users can easily provide feedback by sending messages directly to the admin through the web application. This feedback system allows for seamless communication, ensuring users can share their thoughts and suggestions effortlessly. Admins can then review and respond to the messages promptly.

2.2.2 Development Requirements

The system relies on an internet connection to ensure the continuous maintenance of system data. This ensures that the system remains responsive and can be accessed on any device. Regarding the platform for running and building this web application, the following components are utilized:

- Django Python
- MySQL Workbench
- Visual Studio

Chapter 3

Requirement Analysis

3 Requirement analysis

3.1 Use Case Diagram

A use case diagram serves as a blueprint outlining the anticipated functionalities of a software program. It articulates the expected behavior of the software without delving into technical intricacies. Use cases are articulated in both textual and visual formats, with use case diagrams being a prominent example.

The fundamental concept behind utilizing a use case diagram is to design a system with a user-centric perspective, elucidating how the system should operate in a manner easily comprehensible to users. By detailing visible system actions, we can effectively communicate the system's functionality and behavior.

3.1.1 Use Case of User

Figure 3.1 shows how customer reserve the space and order the product by interacting with system.

- The customer starts by logging in.
- Then they browse products and select what they want to buy.
- They can add multiple items and adjust quantities before adding them to their cart.
- They can then view their cart and edit it before placing their order.
- After placing the order, they can view their payment options.
- Customers generate the reservation request.
- Finally, they can log out.

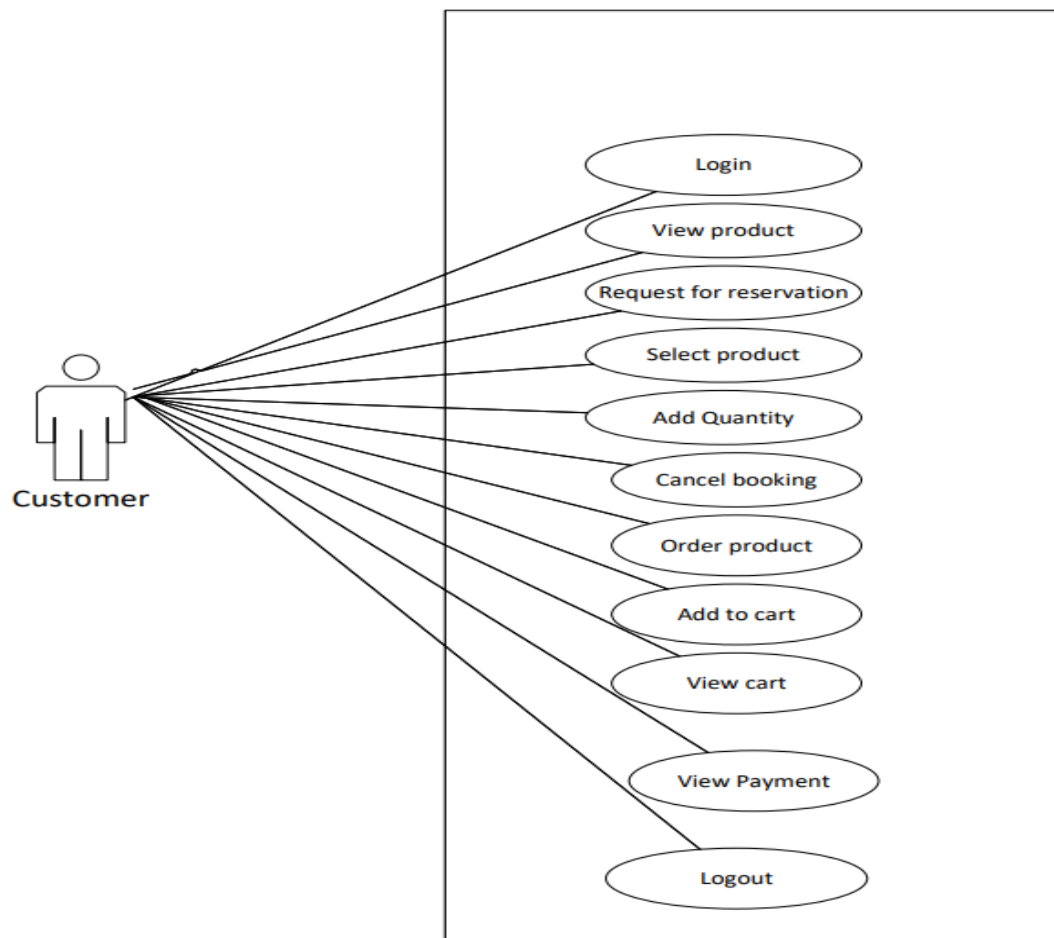


Figure 3. 1 Use Case of Customer

3.1.1.1 Use Case of Reservation

The Table 3.1 below indicates a comprehensive use case template filled in with an example drawn from use case 1 [1].

Table 3. 1 Table of Use case Reservation

Use Case ID	01
Use Case Name	Reserve space
Actors	Customer
Description	This use case allows users to request space reservations by submitting a reservation form. Upon approval, an email notification is sent to the user, providing a confirmation link. Users can also view their reservations on their profile
Trigger	User initiates a space reservation request.

Preconditions	PRE-1. User is logged into the system. PRE-2. Users generate request for reservation.
Post conditions	POST-1. Reservation request is either approved or declined. POST-2. User receives a confirmation email (if approved). POST-3. Reservation details are updated.
Normal Flow	• User logs in. • User navigates to the "Reserve Space" section. • User fills out the reservation form, specifying details such as date, duration, and space requirements. • User submits the reservation request. • System sends a confirmation email to the user. • Reservation details are updated in the system. • User receives the email, which contains a link to confirm the reservation. • User clicks the link to confirm the reservation. • The reservation status is updated to "Confirmed."
Alternative Flows	If the user doesn't confirm the reservation (within 10 days), the system updates the reservation status to "Cancelled."
Exceptions	• If the user is not logged in, they are prompted to log in before initiating a reservation request. • If the user submits an incomplete reservation form, they are prompted to fill in all required details.
Assumptions	The user has the necessary operating environment specifications, and he/she knows the basis and flow of using the app. • The system has email notification capabilities. • User profiles are already set up. • Users are responsible for confirming their reservations via email links. • Users can view their reservation status and details on their profiles.

3.1.1.2 Use Case of Order Product

The Table 3.2 below indicates a comprehensive use case template filled in with an example drawn from use case 1

Table 3. 2 Table of Use case Order Product

Use Case ID	02
Use Case Name	Order Product
Actors	• Customer
Description	This use case allows customers to place orders for products, view cart and pay online or cash on delivery.
Trigger	Customer initiates an order for a product.
Pre-conditions	PRE-1. Customer is logged into the system. PRE-2. Add items into the cart PRE-3. View the cart PRE-4. Product catalog is available in the system.
Post conditions	POST-1. Order is placed and stored in the system. POST-2. Customer receives order confirmation. POST-3. Users may log out
Normal Flow	• Customer logs in. • Customer navigates to the "Order Product" section. • Customer browses the product catalog. • Customer selects a product, specifying the quantity and any customizations. • Customer adds the product to the cart. • Customer reviews the order in the cart and proceeds to checkout. • Customer confirms the order and provides delivery details. • Customer submits the order. • Customer receives an order confirmation email.
Alternative Flows	If the customer cancels the order during the checkout process (Step 7), the order is not placed.
Exceptions	If the customer is not logged in, they are prompted to log in before initiating an order. If the selected product is out of stock, the system alerts the customer and prevents order placement.

Assumptions	The user has the necessary operating environment specifications, and he/she knows the basis and flow of using the app. The system maintains a product catalog with up-to-date information. Customers have registered profiles. Admins have the authority to process orders. Email notifications are used for order confirmation. Product availability is updated in real-time to prevent out-of-stock orders.
---------------------------	---

3.1.2 Use Case of Admin

The Figure 3.2 shows the process of admin manages various tasks. Following is the description of below diagram

- **Login:** The admin user logs in to the system.
- **Signup:** (Optional) A new admin user can sign up for the system.
- **View product:** The admin user can view details of a specific product.
- **Manage product details:** The admin user can add, edit, or delete product information.
- **Cancel order product:** The admin user can cancel an order for a specific product.
- **Accept / Reject reservation:** The admin user can accept or reject a reservation for a product.
- **Manage payment details:** The admin user can add, edit, or delete payment methods.
- **Manage customer data:** The admin user can view, add, edit, or delete customer information.
- **Logout:** The admin user logs out of the system.

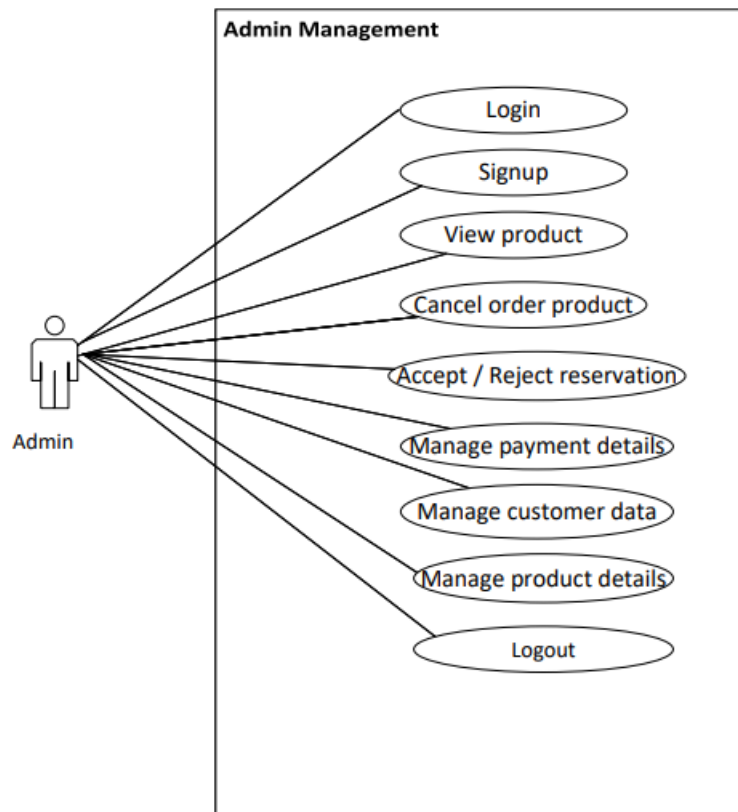


Figure 3. 2 Use case of Admin

3.1.2.1 Use Case of Admin

The Table 3.3 below indicates a comprehensive use case template filled in with an example drawn from use case .

Table 3. 3 Use case description of Product detail

Use Case ID	03
Use Case Name	Product detail
Actors	Admin
Description	Admin manages the product details. Admin was able to add new product category and product type.
Trigger	

	Admin logs in and accesses the admin management section.
Preconditions	• Admin is logged into the system.
Post conditions	• System records admin's decisions and updates order product status.
Normal Flow	• Admin logs into the system. • Admin navigates to the "Admin Management" section. • Admin open the product section. • Admin easily to update the product category and product type.
Alternative Flows	• If the admin needs more information to make a decision for any request (Steps 4, 6, 8), the admin may contact the user or request additional details before proceeding. • If there are technical issues preventing the admin from accessing or reviewing requests (Steps 3, 5, 7), the admin may report the issue to the system administrator.
Exceptions	If the admin is unable to access the admin management section due to login issues, they should contact the system administrator for assistance.
Assumptions	• Admins have the necessary permissions and training to manage product details. • The system provides clear and intuitive interfaces for admin management. • Admin actions are logged and auditable for accountability. • Users are notified promptly of admin decisions regarding their requests.

3.2 Functional requirements

Functional Requirements are identified as crucial elements for the system, originating from end-user inputs and market demands. These specifications outline the expected behavior of the system in specific scenarios, enumerating the features and functions that developers must integrate into the web and app solutions. Precision in articulation is paramount, serving both the development team and stakeholders. Functional requirements essentially encapsulate the

application's features, which play a pivotal role in facilitating user tasks. Functional requirements are the features of our application which helps the user to complete their task. This system will have the following modules:

- Customer must be register before booking the space for storing their product.
- Customer can select the items.
- Users' login the system by inserting their username and password
- They can reset their password.
- Users create their new account.
- They will be capable of changing their passwords.
- Customer must be register before booking the space for storing their product.
- Customer can select the product, product type and sub type, product quantity.
- Admin can accept or decline the request of customer.
- Admin send message the customer about the space allocation.
- Customer must be registered before ordering the product.
- Customers view the details of the product.
- Customer can select the product and quantity of product.
- They can also fill the form of their personal information.
- Admin able to view the ordering details of customer.
- The system provides a unique ordering confirmation number for all successfully executed orders.
- Admin register the new products.
- Admin update the product details in the list.
- Admin view the specific details of product.
- Admin can see the ordered or non-ordered product.
- Customer can select the product in the list.
- Customer can view the product.
- Users can provide feedback about the application.
- Admin update the record of customer in payment list.
- Admin save all the modification in payment.
- Admin can see the specific customer payment detail.
- Customers view their payment details for booking rooms and use the transportation facility.

- Customer will be able to give payment by cash on delivery or pay online.
- All users must use the logout to exit from the application.

3.3 Non-functional requirements

Nonfunctional requirements are not related to the services provide by the system. It is the constraints that imposed on the implementation of the system such as the quality, reliability, and security, etc. We can say some extra conditions and requirements that are not included in the use cases. These are usually called non-functional requirements; some of these are given below [5].

3.3.1 Usability

The application must have a simple and intuitive user interface that is easy to use for both customers and owner. It should also be visually appealing and customizable.

3.3.2 Performance

The application should be fast and responsive, with quick load times and minimal lag. It should also be optimized for different screen sizes and device specifications.

3.3.3 Reliability

The application should be reliable, with minimal downtime and errors. It should be able to handle many users simultaneously without crashing or slowing down.

3.3.4 Availability

This application will be readily available to all its users all the time.

3.4 System Requirements

These requirements consist of the hardware and software components of a computer system that are required to develop and install to use the software efficiently.

3.4.1 Hardware Requirements

- Minimum 4 GB RAM is required to run the application.

3.4.2 Software Requirement

- Internet is necessary to run this application.
- Web server, application server and database will be required to operate the application.

Chapter 4

Design and Architecture

4 Design and Architecture

4.1 System Architecture

The system's architecture delineates the fundamental components, their interconnections, and the interactions that define their relationships. Software architecture and design are influenced by a multitude of factors, encompassing business strategy, quality attributes, human dynamics, design principles, and the IT environment. Within the realm of architecture, functional requirements serve as guiding principles, separating them from nonfunctional decisions. During the design phase, these functional requirements are translated into concrete implementations. The system's accessibility is designed to be user-friendly, allowing anyone to sign up, log in, explore the main page, access information, and gain insights into the system's functionality. The system is modular in nature, with various components working in tandem to deliver a cohesive user experience. To visually articulate and elucidate the architecture, flowcharts can be employed effectively. Figure 4.1 show how Model-View-Template (MVT) architectural module used into project which significantly enhanced the system's structure and functionality.

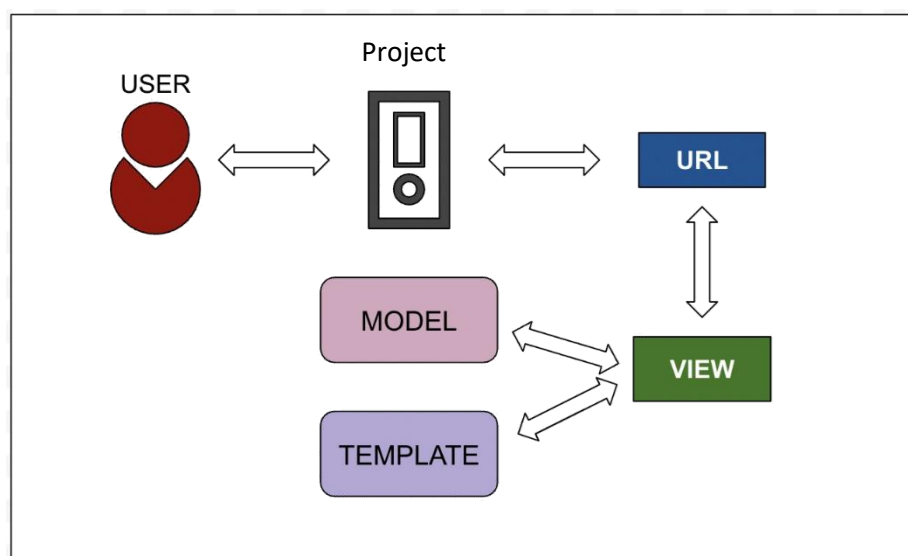


Figure 4. 1 Structure of MVT

MVT, an evolution of the more widely known Model-View-Controller (MVC) architecture, delineates the system into three interconnected components: Model, responsible for managing data and business logic; View, handling the presentation layer for user interaction; and Template, governing the generation of dynamic content and ensuring a seamless integration between Model and View. The MVT architecture not only fosters modularity and code reusability but also streamlines the development process by providing a clear separation of

concerns. This strategic utilization of MVT has proven instrumental in achieving a well-organized and scalable system, offering improved maintainability and ease of future enhancements. Additionally, we will thoroughly explore the merits and advantages of the selected methodology, highlighting how it enhances our project's prospects for success [6].

4.2 Data Representation

4.2.1 Sequence Diagram

The sequence diagram is a visual representation of the interactions between the different objects and components of the system. It shows the flow of messages between the objects and how they collaborate to achieve specific tasks in the system. The sequence diagram captures the dynamic behavior of the system and is useful in identifying potential issues and optimizations in the system's design. Figure 4.2 shows the sequence of actions which system performs.

4.2.1.1 Sequence Diagram of Order Product

Following diagram provides a visual representation of the interactions and chronological flow of events involved in the process of placing an order for a product. Figure 4.2 the sequence diagram you sent me shows the process of a customer ordering a product online. Here's a breakdown of the steps:

- **Login:** The customer logs in to the website.
- **View Products:** The customer browses the products on the website.
- **Select Product:** The customer selects the product they want to order.
- **View Product Details:** The customer views the details of the selected product, such as the price, description, and reviews.
- **Add to Cart:** The customer adds the product to their shopping cart.
- **View Cart:** The customer can view the contents of their shopping cart at any time.
- **Place Order:** The customer proceeds to checkout and place their order.
- **Apply Coupon:** The customer can enter a coupon code if they have one.
- **Select Delivery Option:** The customer can select their preferred delivery option.
- **Enter Payment Information:** The customer enters their payment information.

- **Payment Confirmation:** The system confirms the payment.
- **Order Confirmation:** The customer receives an order confirmation email.

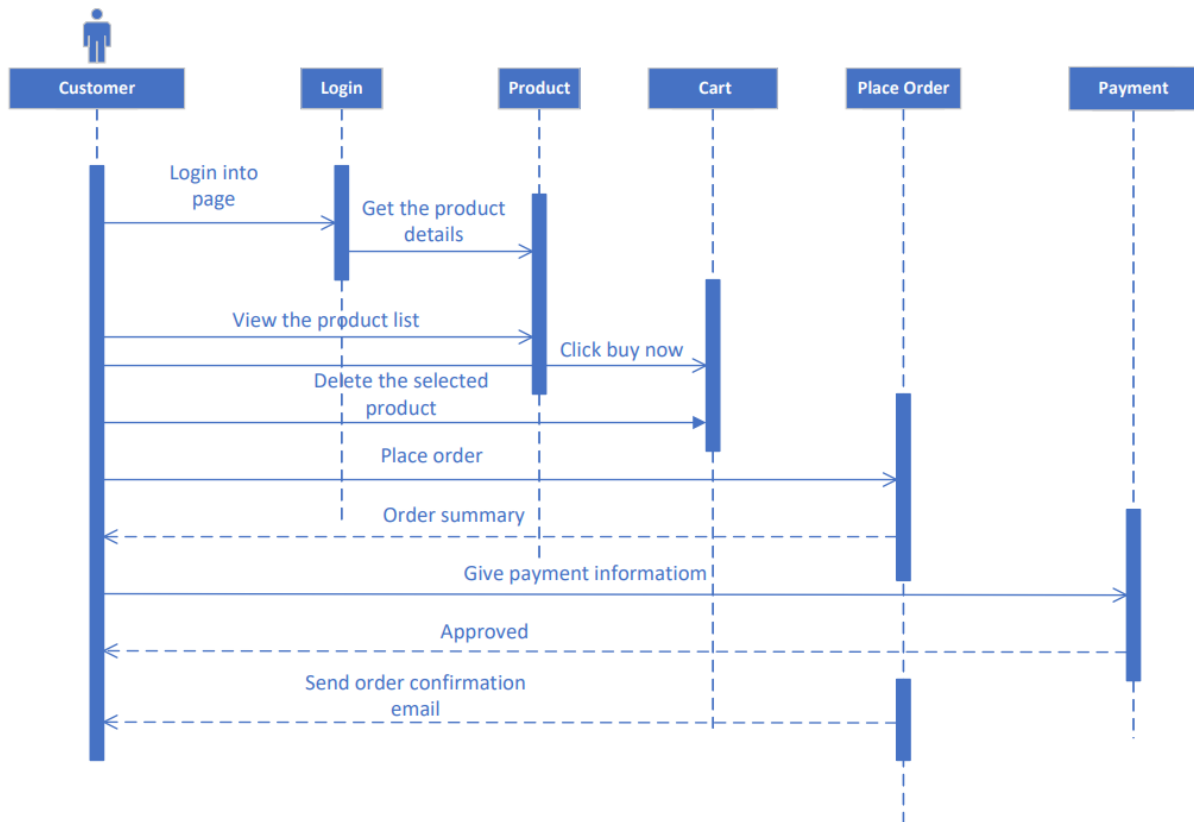


Figure 4. 2 Sequence diagram of Order Product

4.2.1.2 Sequence Diagram of Reservation

Following sequence diagram provides a step-by-step depiction of the interactions between the user and the system in the process of reserving potatoes and generating a bill summary. Figure 4.3 depicts the process of a customer making a reservation, likely within a system for booking appointments, events, or services.

- **Login to system:** The customer initiates the process by logging into the system.
- **Get the details of reservation:** The system retrieves relevant reservation details, likely based on customer input or previous selections.
- **Request for Reservation:** The customer formally requests the reservation.

- **Get the information from request:** The system gathers necessary information from the request.
- **Confirm Reservation:** The system confirms the reservation and completes the process.
- **Bill summary show:** A bill summary is displayed, presumably indicating payment details or reservation confirmation.

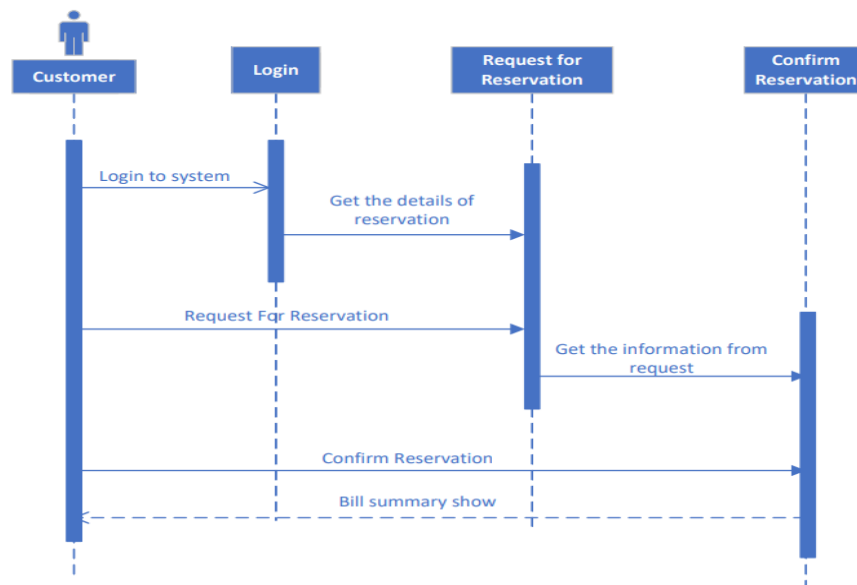


Figure 4. 3 Sequence of Reservation

4.2.1.3 Sequence Diagram of Admin Manages Reservation

Figure 4.4 shows how the admin manages the system and interaction between different parts of a Keep & Lock system when a customer makes a reservation, with a focus on managing room allocations and order product. Here's a breakdown of the steps:

- The process starts with a Search for Room where the customer enters their desired dates, number of guests, and preferred room type.
- Based on this information, the system then checks for Room Availability.
- If a room is available, the system displays the Room Details to the customer.
- The customer can then choose to Reserve Room or go back to Search for Room if they're not happy with the available options.
- If the customer chooses to reserve the room, they will need to confirm their Guest Information and enter their Payment Details.

- Once everything is confirmed, the system will process the reservation and send a Confirmation Email to the customer.
- If a room is not available for the customer's requested dates, the system will display a Message informing them of this.
- The customer can then choose to:
 - Change their dates or search criteria and go back to Search for Room.
 - Join a waitlist for the desired room.
- The process ends once the customer has either successfully reserved a room or chosen not to proceed further.

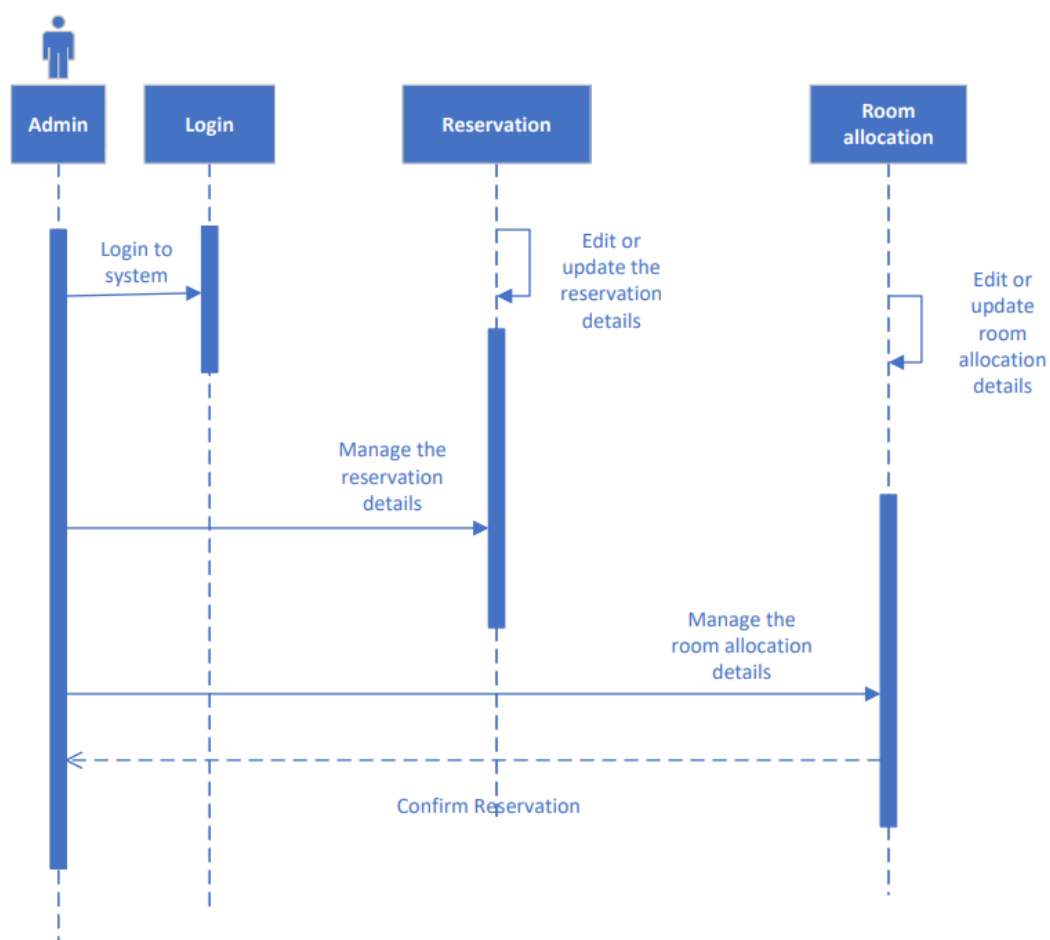


Figure 4. 4 Sequence of Admin Manages Reservation

4.2.2 Class Diagram of Keep & Lock

The class diagram is a graphical representation of the system's classes, their attributes, and the relationships between them. It depicts the static structure of the system and shows how the different classes in the system are related to each other. Figure 4.5 models a system involving customers, reservations, orders, and payments.

- **Customer:** Represents a customer with attributes like name, email address, phone number, and address.
- **Login:** Manages login-related information, including member name, login ID, username, password, and functions for logging in, creating accounts, resetting passwords, etc.
- **Admin:** Represents an administrator with attributes like ID, name, phone number, email address, and functions for modifying, inserting, deleting, and viewing details.
- **Reservation:** Represents a reservation with attributes like ID, customer ID, name, date and time, status, content, and functions for confirming, canceling, and managing reservations.
- **Order:** Represents an order with attributes like ID, customer ID, name, time, employee ID, status, total price, and functions for accepting, canceling, confirming, and managing orders.
- **Payment:** Represents a payment with attributes like ID, date and time, amount, type, details, and functions for getting information, setting information, updating, and checking status.

Relationships:

- **One-to-one:** Admin and Login have a one-to-one relationship, indicating each admin has a unique login.
- **One-to-many:** Customer has a one-to-many relationship with Reservation and Order, meaning a customer can have multiple reservations and orders.
- **One-to-one:** Order has a one-to-one relationship with Payment, suggesting each order has a corresponding payment.

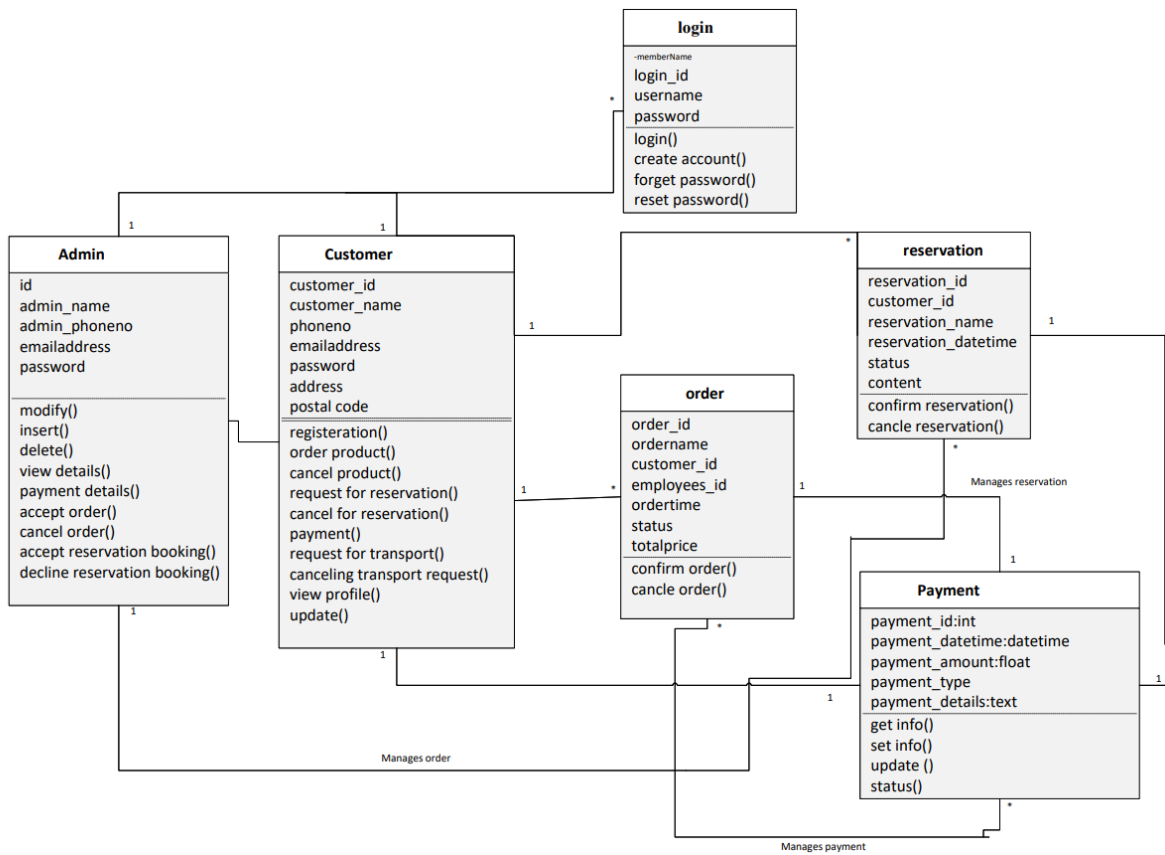


Figure 4. 5 Class diagram of Keep & Lock

4.2.3 Data Flow Diagram

This diagram represents the flow of data or process of the system. It gives complete information about the outputs and generated after providing user's inputs of each module and process itself.

LEVEL 0:

- DFD level 0 is also called context level diagram.
- DFD level 0 has only one process that is the whole system.

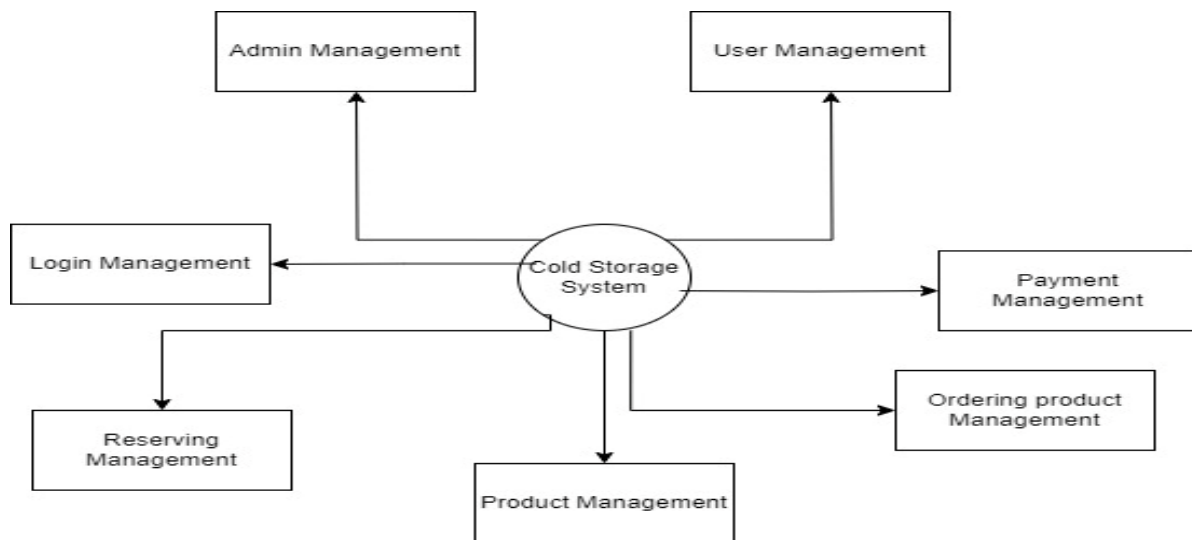


Figure 4.6 DFD level 0 for Keep & Lock

LEVEL 1:

- In Figure 4.7 show a main process has been sub divided into sub processes.

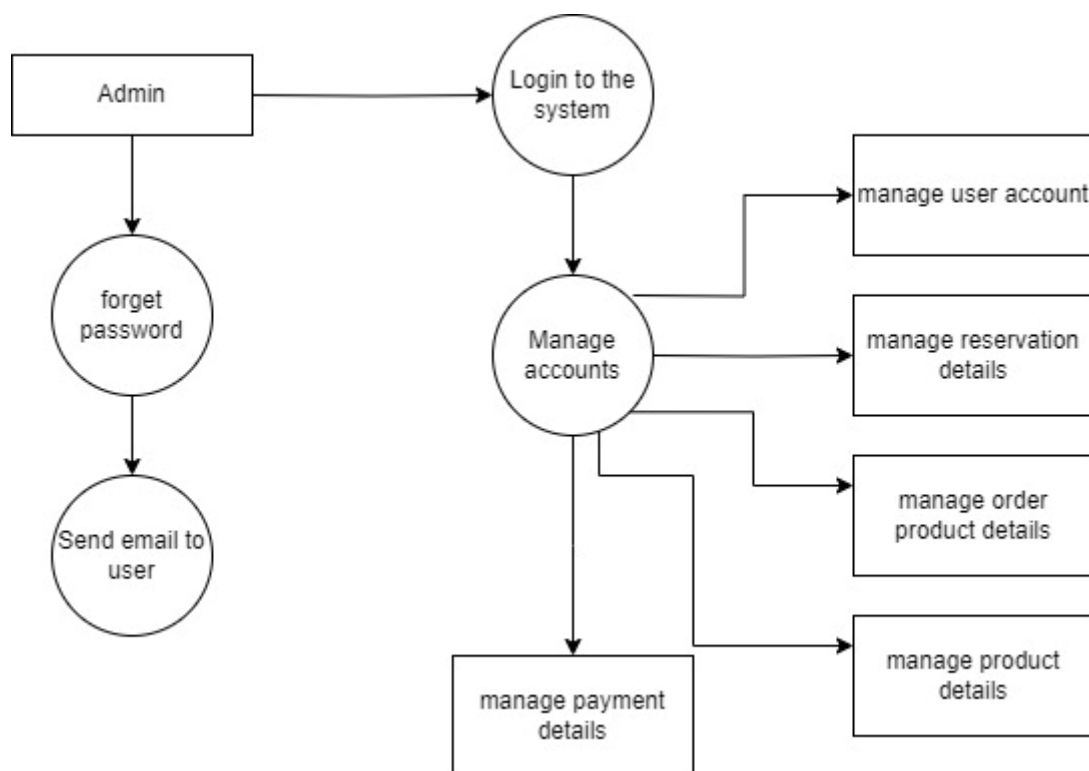


Figure 4.7 DFD level 1 for Keep & Lock

4.2.4 Process Flow Representation

Process flow is the graphical representation of the system to describe the processes. It consists of tasks and their order. Process flow helps assist with brainstorming and communication with the process design. Figure 4.8 and Figure 4.9 helps to understand the flow of the system which helps the developer to design a system.

- **For Admin Console**

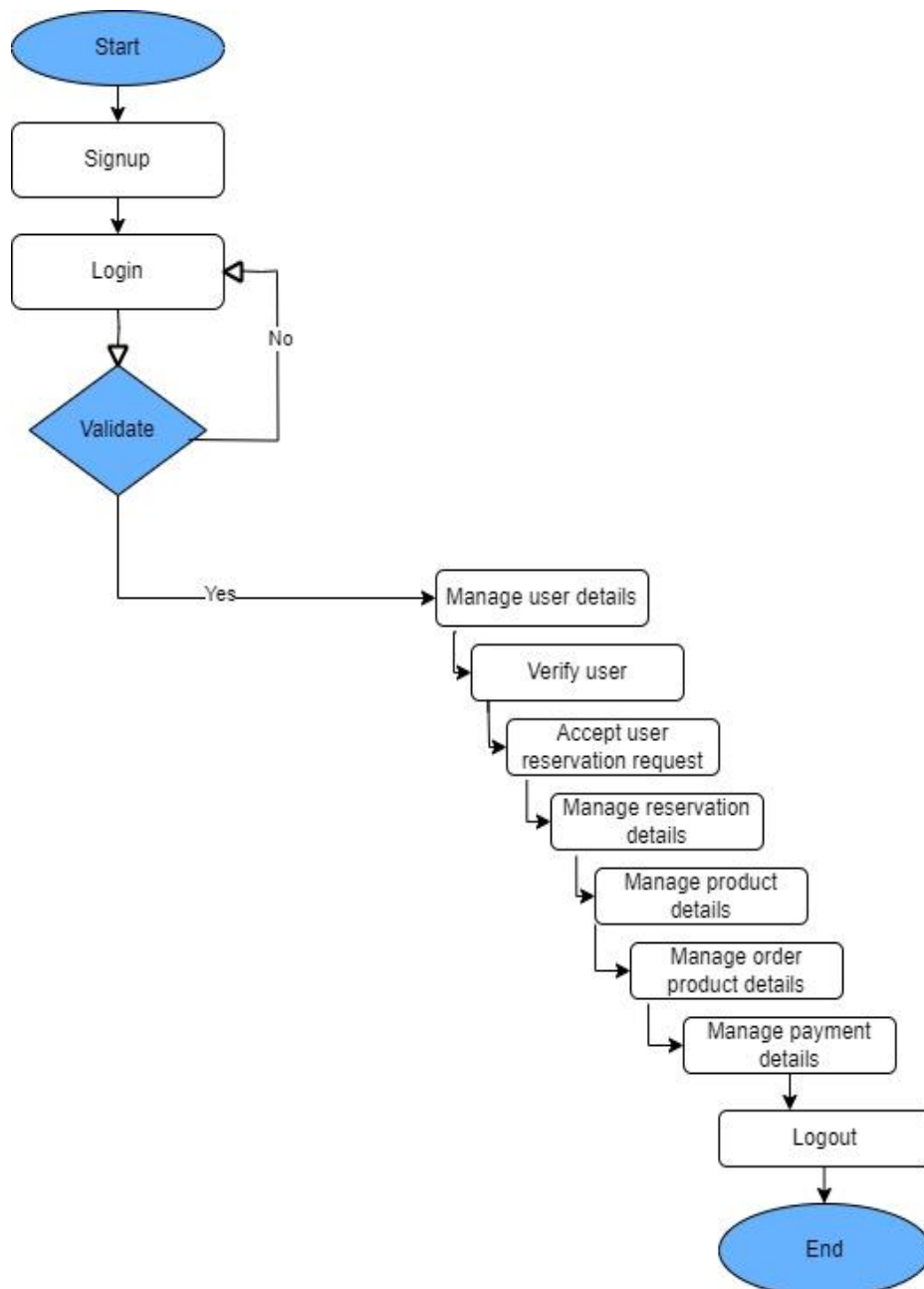


Figure 4.8 Process flow diagram of Admin Console

- **For User Console**

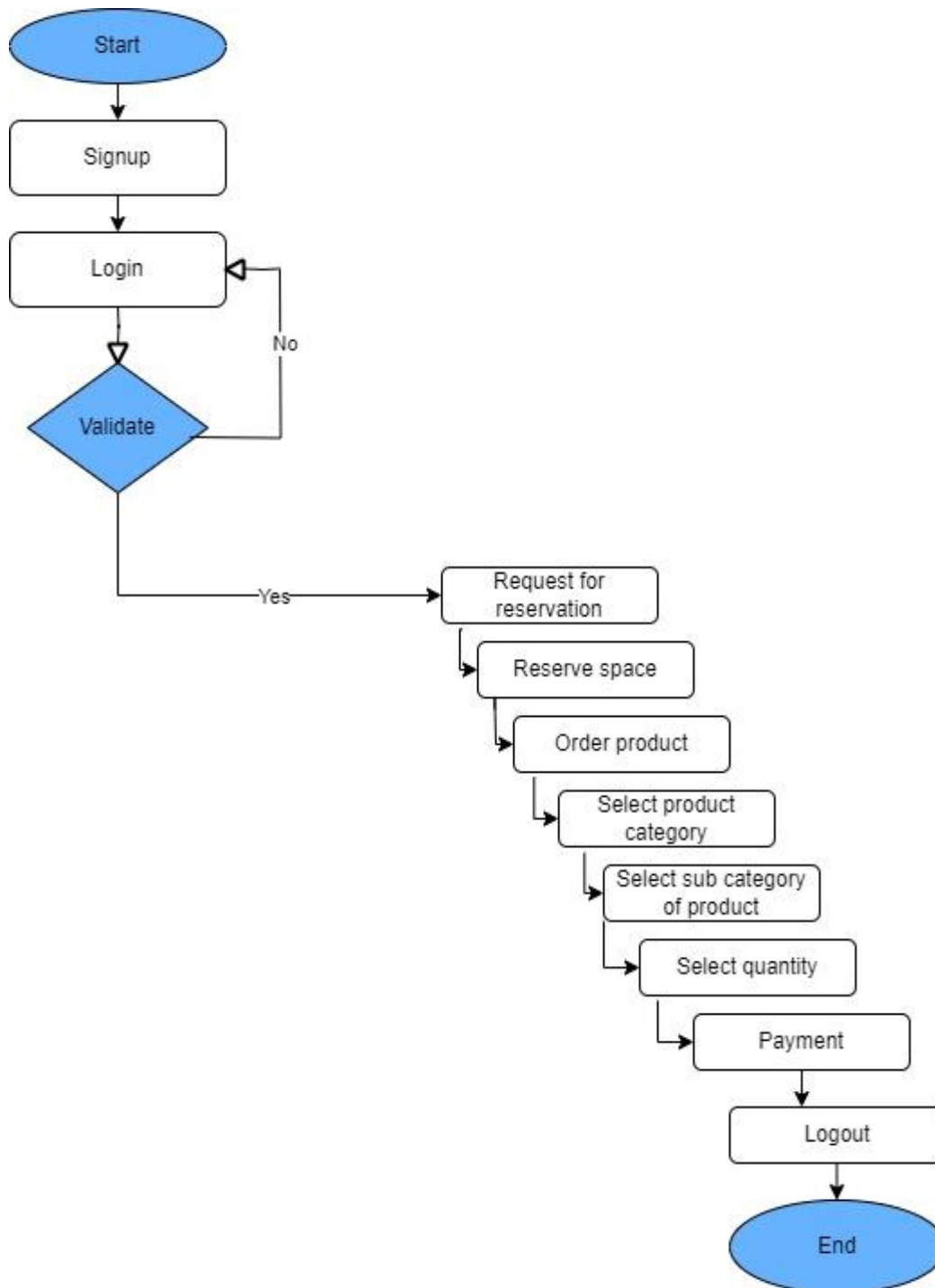


Figure 4.9 Process flow diagram of Customer Console

4.2.5 Entity Relationship Diagram

People also call these types of diagrams ER diagrams and Entity Relationship Models. An ERD visualizes the relationships between entities like people, things, or concepts in a database. An ERD will also often visualize the attributes of these entities.

4.2.5.1 ERD Diagram of Order Product:

Figures 4.10 shows the order relationship with admin and customer.

Entities:

- **Customer:** Represents the customer placing the order. Attributes include customer ID, name, email address, and phone number.
- **Product:** Represents a product available for purchase. Attributes include product ID, name, description, price, and image URL.
- **Order:** Represents an individual order placed by a customer. Attributes include order ID, customer ID, date, status (e.g., pending, processing, shipped, delivered), and total amount.
- **Order_Item:** Represents an item within an order. Attributes include order item ID, order ID, product ID, quantity, and price.

Relationships:

One-to-many: A customer can place many orders (one-to-many).

One-to-many: A product can be included in many orders (one-to-many).

Many-to-many: An order can contain many products (many-to-many) through the **Order_Item** entity, which acts as an intermediary table to specify quantity and price for each product within an order.

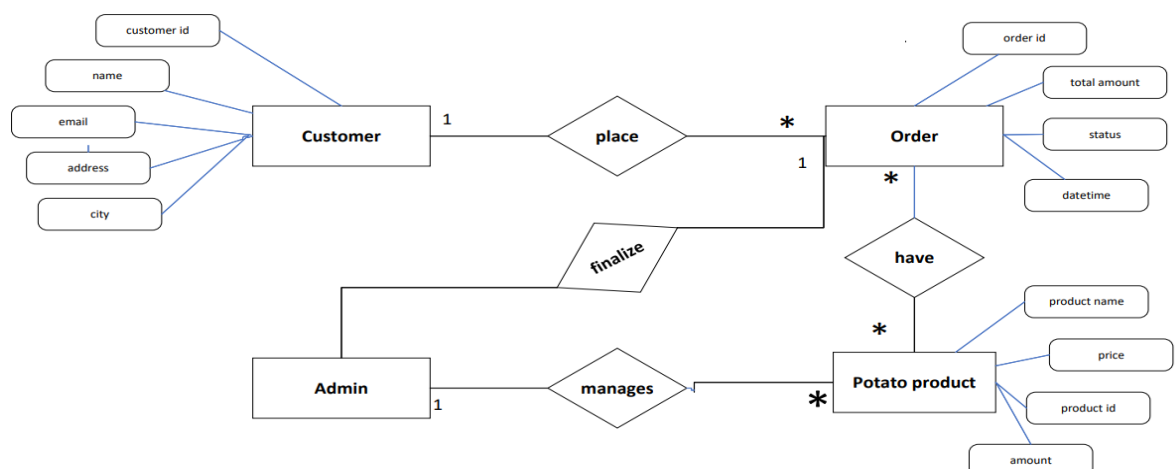


Figure 4. 10 ERD Diagram of Order Product

4.2.5.2 ERD Diagram of Reservation:

The Figure 4.11 has two main entities: Customer and Potato Product.

- Customer has attributes like customer id, name, email, address, and city.
- Potato Product has attributes like product id, product category, product type, and bory weight.

There are also two relationships between these entities:

- Customer can place Reservations for Potato Products. This relationship has attributes like reservation id, datetime, and total amount.
- Admin can manage Potato Products. This relationship has an attribute called have.

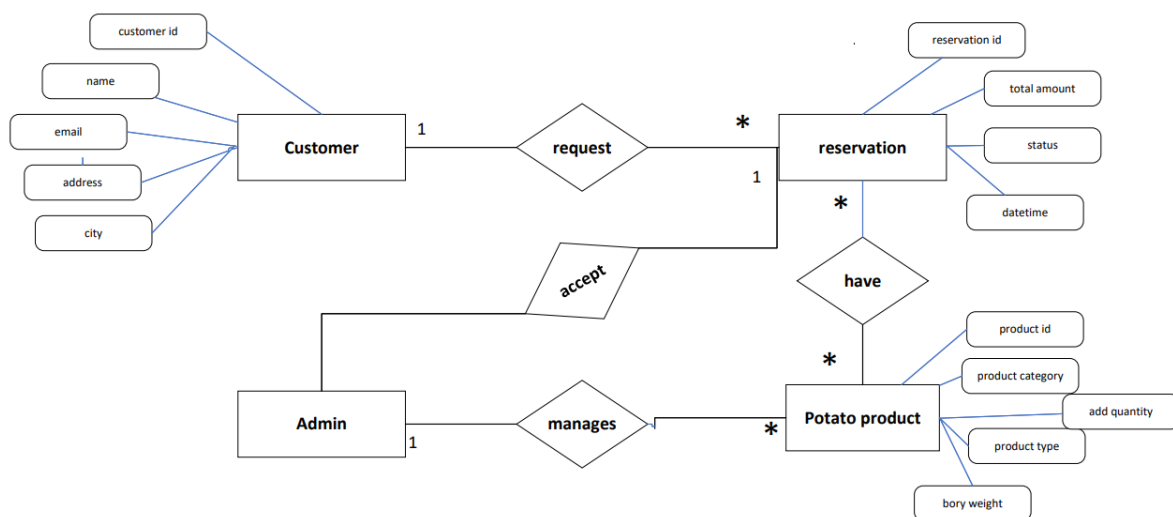


Figure 4. 11 ERD Diagram of Reservation

4.3 Design Models

4.3.1 Rapid Application Development

Rapid application development model is used to develop this application because it gives importance to prototyping. RAD is an object-oriented methodology. It also helps to get rapid feedback by delivering the modules of system in incremental form. It helps the developers to make the changes or update the system easily without starting from the beginning. It is more flexible to develop the web based cold storage system.

This helps to reduce the risk of errors and omissions in the final product, as any issues can be identified and corrected early in the development process. Overall, the use of the rapid application development model has helped to ensure that the keep n lock web-based application is developed efficiently, with a focus on producing a high-quality product that meets the needs of its users. Figure 4.4 shows the rapid application development model of a system.

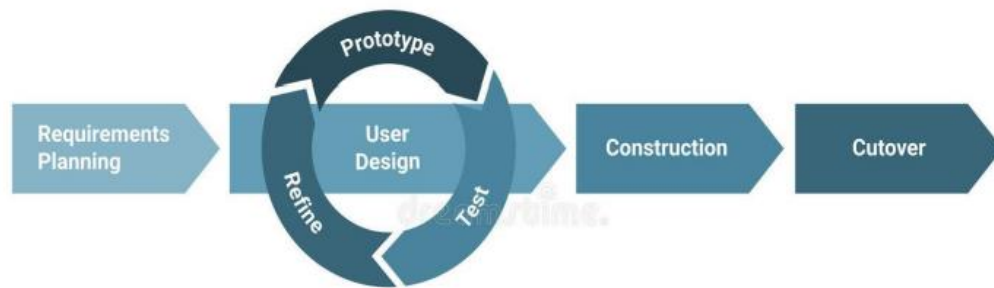


Figure 4.10 Rapid Application Model

Chapter 5

Implementation

5 Implementation

In this chapter, we'll focus on the implementation of the “Keep & lock” web application. The most important goal of this phase is to develop the application. The work in this phase should be much more straightforward because of the work done in the planning and design phases. This phase involves changing design specifications into executable programs. When the design is there, developers can have an idea of the looks of the application. All that is needed by developers is to put them in one place to understand.

- The Model-View-Template (MVT) framework, an evolution of the widely known Model-View-Controller (MVC) architecture, organizes web application code into three essential components: Model, View, and Template. The Model manages data and business logic, ensuring information integrity. The View handles user interaction and information display, while the Template dynamically generates content, seamlessly integrating the Model and View. Noteworthy is MVT's focus on decoupling, enabling developers to modify or replace individual components without impacting the entire system, contributing to code modularity and maintainability.
- A workbench database plays a crucial role in website development by serving as the backbone for storing and managing data essential for web applications. It provides a centralized repository for information such as user profiles, content, product listings, and more, allowing developers to efficiently retrieve, update, and display data on the website.

5.1 Modules of the Keep & Lock

The modules of Keep & Lock is as follow.

5.1.1 Sign-up Module

The user enters their credentials on the sign-up page, typically find fields to input login credentials. These usually include:

- Username: Enter your unique username.
- Email: Enter your email address associated with your account.
- Password: Create a strong and secure password following the app's password requirements (e.g., length, special characters, etc.).
- Confirm Password: Re-enter the password to confirm accuracy.

5.1.2 Login Module

Access the Login Page: Visit the web app's homepage and locate the "Log In" or "Sign In" button/link. Click on it to access the login page.

Provide Credentials:

- Username/Email: Enter the username or email address associated with your account.
- Password: Input your password. Ensure its accurate and case sensitive.
- Authentication: Click the "Log In" or "Sign In" button to submit your credentials for authentication.

5.1.3 Reserve Module

The user can reserve space for storing potatoes etc. The user can easily check that how much space is free for reserving. For reserving the web application required some credentials such as:

- The user firstly enters the web page and select product type like potatoes, tomatoes and etc.
- After selecting product, potatoes field contain some categories of product such as rashan and seeds.
- The user must select subcategory of potatoes from following options like white, red, mozika and sangitta.
- The user also selects quantity of potatoes minimum 50 bags, then submit request.
- The web app also has online billing convenience for user.

5.1.4 Ordering Module

- The user also order product from this web application.
- Select product type and category of product then select quantity of product minimum 50 bags.
- In ordering module there are also billing and payment convenience

5.2 External API's

Following Table 5.1 describe the API that was used in our project.

Table 5. 1 This table of External API

API	Description	Usage in Django Project
Google API	The Google API allows integration with various Google services and provides OAuth for user authentication.	Implement OAuth2 authentication with python-social-auth or django-allauth to enable Google login. Use the Google API to fetch user details and profile information after authentication.
Facebook API	The Facebook API facilitates integration with Facebook's features, including user authentication via OAuth.	Integrate OAuth2 authentication using python-social-auth or django-allauth for Facebook login. Utilize the Facebook Graph API to retrieve user information and profile data post-login.
Stripe API	The Stripe API is a payment gateway that enables secure online transactions and financial processing.	Integrate Stripe API for processing online payments in a Django project. Use the API to handle payments, subscriptions, and manage financial transactions securely.

5.3 User Interface

The application's interface will be designed to be straightforward and intuitive, ensuring ease of use for the user. The user will be able to navigate and utilize the application independently without relying on external assistance. Some pages of the user interface are further explained along with images below:

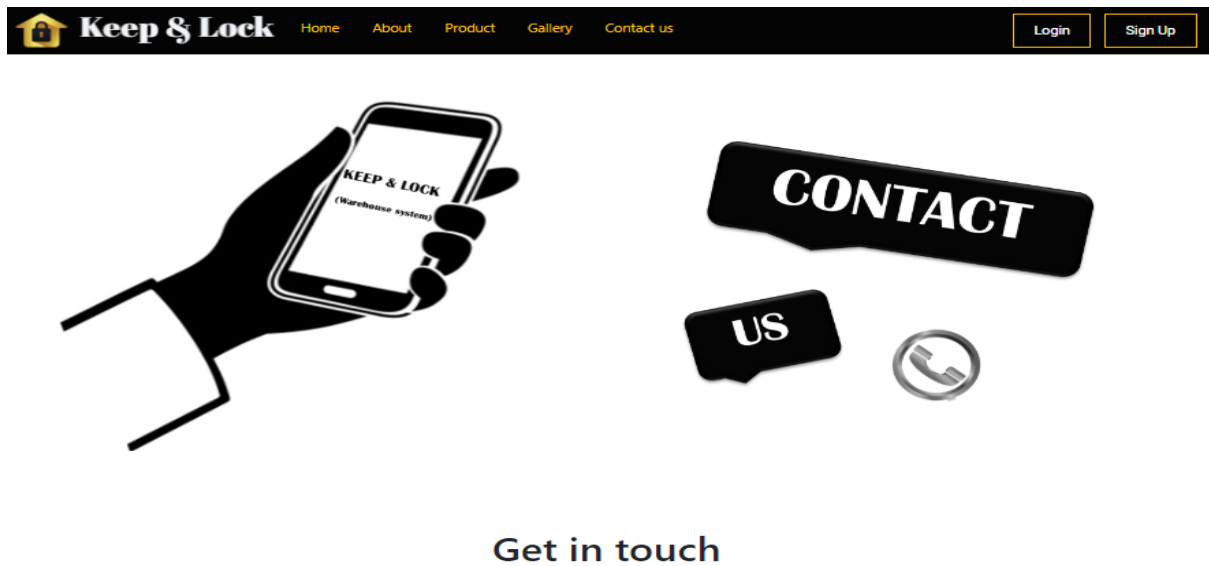


Figure 5. 1 Interface Contact Us

Figure 5. 1 Contact us show a Contact Us form, fostering a user-friendly experience and facilitating efficient communication between users and the website's support or administrative team.

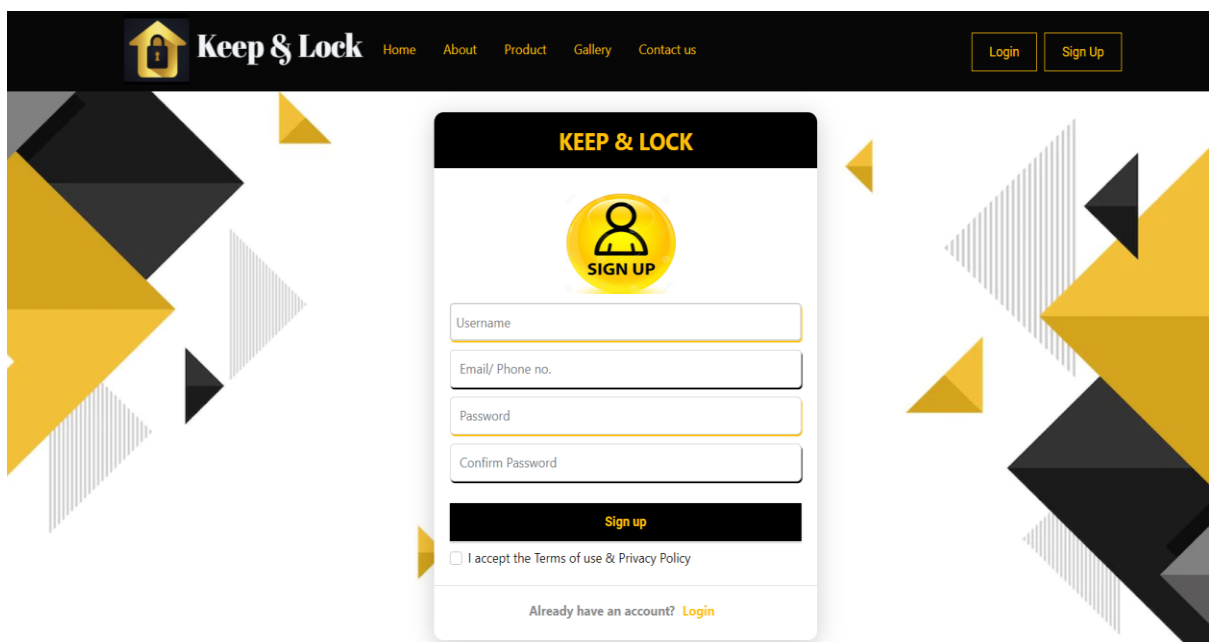


Figure 5. 2 Interface of Sign up

Figure 5.2 show this sign-up form has username which select a unique username for your account identity. Email & Phone which is used to enter contact details for account verification. While Address & Password provides your address and set a secure password.

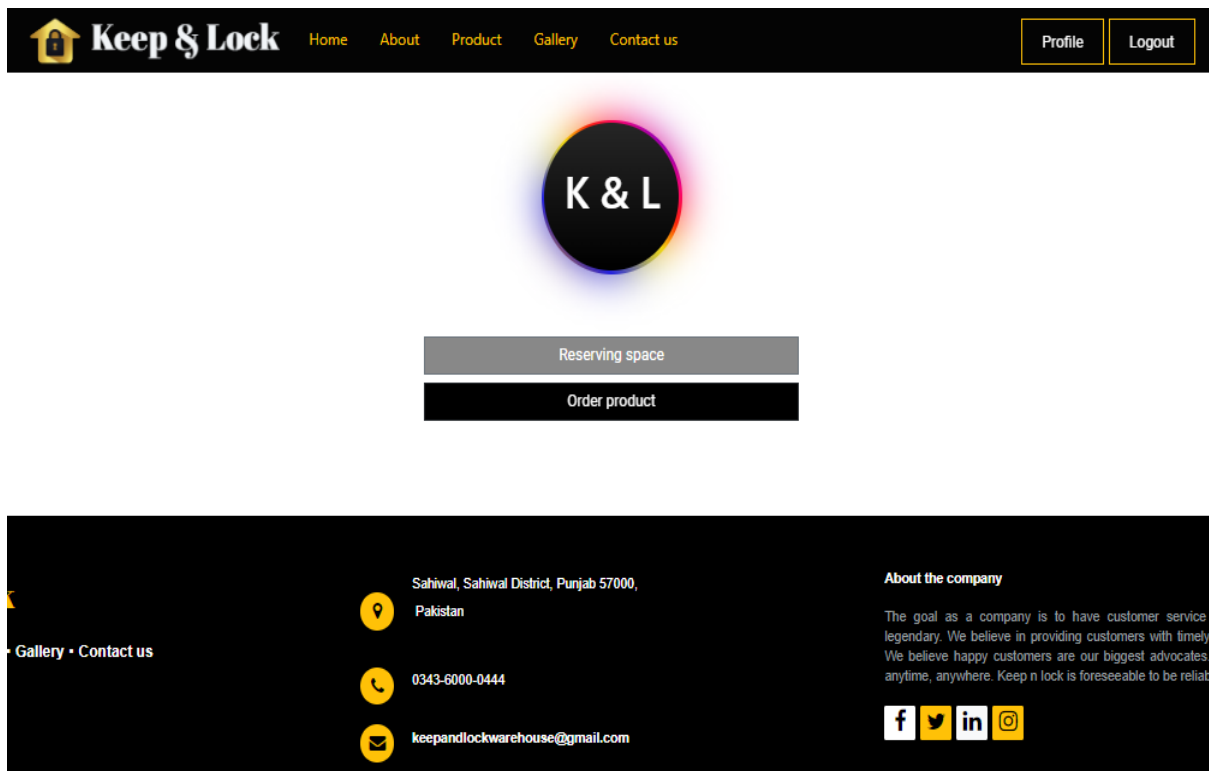


Figure 5. 3 Selection of reserving & order product

Figure 5.3 show this interface used when user want to either booking their reservation or order the product

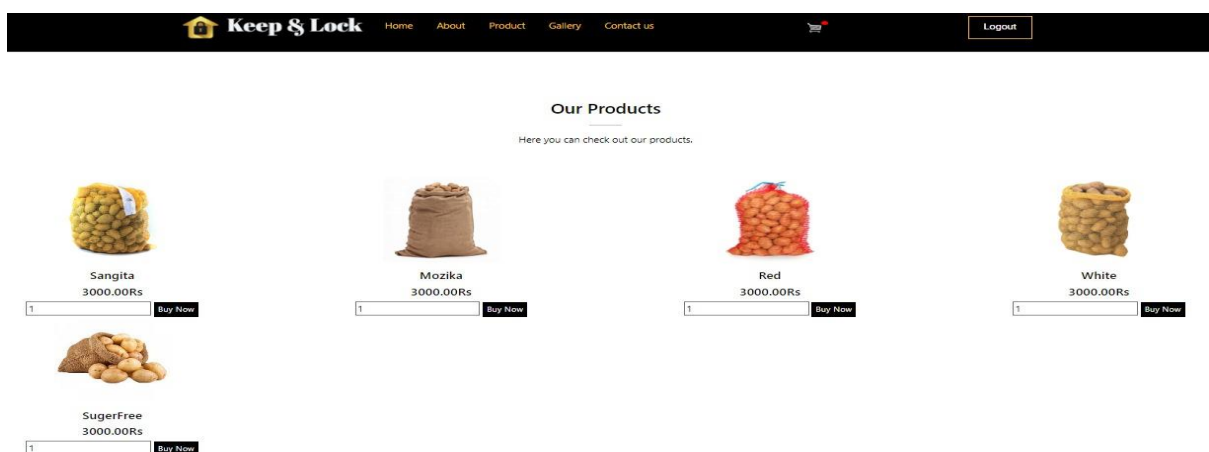


Figure 5.4 Buy now page

Figure 5.4 show this template shows the different product items having the option of buy now or add to cart


[Home](#)
[About](#)
[Product](#)
[Gallery](#)
[Contact us](#)
[Logout](#)

User Information:

Name:

Email:

Shipping Information:

Address:

City:

State:

Zipcode:

Country:

Select a payment method:

☒ Cash on Delivery
☐ Pay Online

Place Order

[← Back to Cart](#)
Order Summary

	Sangita	3000.00 RS	x3
	Mozika	3000.00 RS	x5
	Red	3000.00 RS	x1

Items: 3
Total: 27000.00 Rs

Figure 5.5 Checkout

This Figure 5.5 show this form able the user to place an order and also having the different option of payment such as cod or pay online


[Home](#)
[About](#)
[Product](#)
[Gallery](#)
[Contact us](#)
[Profile](#)
[Logout](#)

Select Product , Save Time

Select the product to keep in cold storage

Request Reservation Form

Customer name:

E-mail:

Phone no#:

Address:

#	Product	Product items	Product sub_items	Quantity	Potato Bag Weight	
1						

Request for Reservation

Figure 5. 6 Interface of Reservation

Figure 5.6 show this form ables the user to request the admin for booking space for their potatoes

5.4 Screen Objects and Actions

The keep & lock application has a variety of screen objects and actions that allow users to interact with the application and perform various tasks. The main screen of the application features a navigation bar with buttons for accessing different modules, such as contact us, login, Sign up, Order product and reserving space. Each module has its screen with specific screen objects and actions.

For example: The signup module allows user to sign up into the system by entering their username, email and password with confirmation.

The login module allows user to login into the system and use it by entering their valid email address and password which they set when sign up in the system.

The reserving module allows user to reserve space for storing products by selecting product type, product category and sub-category, adding the quantity of product. After reservation this module passes to billing phase and duration of storing and give online payment option.

The ordering module allows user to order product from web application by entering product type, category and quantity of product. Adding more, it also contains billing and payment sequence.

Moreover, the app features various input fields, such as text fields that allow users to enter and select relevant information. The app also includes buttons for submitting forms and for performing various actions, such as reserving form, ordering form, billing and payment method. The overall design and layout of the app aim to provide a user-friendly and intuitive experience for users, allowing them to easily navigate the app and access the information they need.

Chapter 6

Testing and Evaluation

6 Testing and Evaluation

The testing and evaluation phase is a crucial component of the keep n lock web-based project, ensuring that the developed application meets the desired requirements and is reliable and functional. This chapter outlines the various types of testing that will be conducted during the project, including manual testing, system testing, unit testing, functional testing, and integration testing [7].

6.1 Manual Testing

Manual testing is the process of verifying the functionality of the application manually by the testers. This process is essential to ensure that the application meets the user requirements and performs as expected. The following Table 6.1 shows the test cases and results for manual testing:

Table 6.1 Manual Testing

Test Cases	Expected Result	Actual Result	Pass/Fail
User Login	Successful	Successful	Pass
User Registration	Successful	Successful	Pass
View products	Accurate	Accurate	Pass
Add quantity	Accurate	Accurate	Pass
Add product	Accurate	Accurate	Pass
Order Product	Accurate	Accurate	Pass
Request reservation	Accurate	Accurate	Pass
Reserving space	Successful	Successful	Pass
Send message	Successful	Successful	Pass
Application Performance and Response	Smooth	Smooth	Pass
Application Navigation	Easy	Easy	Pass
Application Usability and User Friendliness	High	High	Pass

6.1.1 System Testing

System testing is the process of verifying that the application functions as intended in the overall system environment. The following Table 6.2 shows the test cases and results for system testing:

Table 6.2 System testing

Test Cases	Expected Result	Actual Result	Pass/Fail
Compatibility with web application	Compatible	Compatible	Pass
Compatibility with Device	Compatible	Compatible	Pass
Data Security	Secure	Secure	Pass
Data Backup	Reliable	Reliable	Pass
User Management	Effective	Effective	Pass
Application Response Time	Acceptable	Acceptable	Pass
Application Resource Usage	Optimal	Optimal	Pass

6.1.2 Unit Testing

Unit Testing 1: Signup a user

Testing Objective: To ensure the user can sign up into the system properly.

Table 6.3 Unit testing of sign up

Test Case	Input	Expected Output	Actual Output	Pass/Fail
1	User will sign up through username	E-mail: Raeesamukhtar55@gmail.com Password: Raisa1661	Successfully signup into the system.	Pass

	email and password			
2	User will sign up through username email and password	E-mail: zunairarasheed64@gmail.com Password: 1234zr	Successfully signup into the system.	Pass

Unit Testing 2: Login into the system

Testing Objective: To ensure the login form work properly.

Table 6.4 Unit testing of login

Test Case	Input	Expected Output	Actual Output	Pass/Fail
1	User will login through email and password or direct to Google.	E-mail: sanasiddique6262@gmail.com Password: sana@62	Successfully login into the system.	Pass

Unit Testing 2: Reserved the space

Testing Objective: To ensure the reservation form work properly.

Table 6.5 Unit testing of reserving space

Test Case	Input	Expected Output	Actual Output	Pass/Fail
1.	User will reserve space by selecting product type, category and quantity.	Potatoes Rashan Red 50 bags	Successfully reserve the space	Pass
		Potatoes		Pass

2.	User will reserve space by selecting product type, category and quantity.	Seeds White 120 bags	Successfully reserve the space	
----	---	----------------------------	--------------------------------	--

6.2 Integration Testing

Integration testing is a critical phase in software development where individual components or modules of a software application are combined and tested as a group to ensure they work together seamlessly [8]. Following Table 6.6 show the integration testing of user.

Table 6.6 Integration testing

Test Case name	Signup a user
Test Case number	01
Description	The user can sign up into the system with enter his/her basic information like username, email and valid password.
Testing technique used	Manual testing, Interface Testing using the black box testing technique, Condition Coverage testing.
Precondition	Users should have valid email and password for signup.
Input values	Enter email xyz@gmail.com Enter password 12345
Valid inputs	Enter email xyz@gmail.com Enter password 12345

Steps	• Users need to on their internet services. • User must need valid username, email and password on the dashboard to click on the signup button and enter his/her basic information. The status is signup successfully
Expected output	Sign up successfully
Actual output	Status changed.
Status	Pass

6.3 Functional Testing

Functional testing will be conducted to ensure that the application meets the functional requirements specified in the project scope. The testing will cover all modules of the application, including add quantity, add product. The following Table 6.7 shows test cases for functional testing:

Table 6.7 Functional testing

Test ID	Test Description	Expected Result
F01	Verify that the user can login into the system	The user can login into the system.
F02	Verify that the user can sign up into the system.	The user can correctly sign up into the system.
F03	Verify that the user can reset the password.	The user can reset the password.
F04	Verify that the user can view the product	The user can see the available products.
F05	Verify that the user can add product into the cart.	The user can add product into the cart.
F06	Verify that the user can add quantity of product	The user can easily add quantity of product.
F07	Verify that the user can request for reservation.	The users can easily request for reservation

F08	Verify that the user can confirm their reservation	The user can easily confirm the reservation after getting the confirmation mail from admin.
F09	Verify that the user can view their reservation on profile	The user can easily go to the profile and shows that he/she has successfully reserve space for their potatoes
F10	Verify that the user can receive mail when reservation time limit is finish.	The user can receive the mail when reservation time limit is finish.
F11	Verify that the user can send message.	The user can send message to admin.

Chapter 7

Conclusion and Future Work

7 Conclusion and Future Work

7.1 Conclusion

A web-based cold storage application offers substantial advantages for both customers and cold storage facility owners. Users can easily place orders for products and reserve storage space. The system also proactively notifies customers when they approach their storage capacity limits. Owners benefit from valuable insights into their cold storage operations, including a comprehensive view of registered customers and workforce details, such as staff capacity. They can efficiently track online payments as well. The admin has the capability to monitor the status of the cold storage facility, assess employee performance, and gauge customer engagement. This application not only saves time but also significantly reduces the risk of errors and inaccuracies often associated with manual systems.

7.2 Future Work

We plan to add modules to this application in the future. These modules are:

- Temperature Monitoring
- Camera surveillance

These modules will enhance the functionality of the application, making it more user-friendly and efficient. In system if we integrate a temperature monitoring system into your setup to monitor the temperature of your storage units in real-time. If the temperature fluctuates in any storage unit, you will receive an immediate alert. Camera surveillance can be integrated into your system to enhance security and monitoring capabilities. By adding camera surveillance, you'll have a visual record of activities within your cold storage facility, allowing you to monitor the premises remotely and review footage if any security concerns or incidents arise. This added layer of security can help safeguard your stored products and ensure the safety of your facility.

Overall, adding these modules will provide a more comprehensive and complete user experience and we are excited to implement them soon.

References

- [1] J. Evans *et al.*, “Cold store energy usage and optimisation,” presented at the Proceedings of the 23th International Congress of Refrigeration, 2011.
- [2] A. Pataskar, J. Montenegro Navarro, and R. Agami, “ABPEPserver: a web application for documentation and analysis of substituents,” *BMC Cancer*, vol. 23, no. 1, pp. 1–7, 2023.
- [3] H. Lan, “Web-based rapid prototyping and manufacturing systems: A review,” *Comput. Ind.*, vol. 60, no. 9, pp. 643–656, 2009.
- [4] W. Kulchatchai, “Cold storage and fishery online,” 2004.
- [5] P. Avgeriou, J. Grundy, J. G. Hall, P. Lago, and I. Mistrík, *Relating software requirements and architectures*. Springer Science & Business Media, 2011.
- [6] P. Morville and L. Rosenfeld, *Information architecture for the World Wide Web: Designing large-scale web sites*. O’Reilly Media, Inc., 2006.
- [7] H. Tanno and X. Zhang, “Test script generation based on design documents for web application testing,” presented at the 2015 IEEE 39th Annual Computer Software and Applications Conference, IEEE, 2015, pp. 672–673.
- [8] K. M. Mustafa, R. E. Al-Qutaish, and M. I. Muhairat, “Classification of software testing tools based on the software testing methods,” presented at the 2009 Second International Conference on Computer and Electrical Engineering, IEEE, 2009, pp. 229–233.